

Expected Output:

```
{  
  "orderId": 1001,  
  "items": [  
    { "name": "Biryani", "subtotal": 360 }  
  ],  
  "totalAmount": 400  
}
```

Question 3 – Filter Orders

Route: /filter

Requirements:

- Filter orders by **restaurant name**
- Input should be case-insensitive
- Display only matching orders

Constraints:

- Must use .filter()
- Must not refetch data
- Must use global state

Validation Rules:

- Empty input → show error message
- Invalid restaurant → show “No results found”

Interdependency (Important):

- Filtered results must reflect only **valid orders (Q1 rules)**

Question 4 – Order Status Logic

Route: /orders

Requirements:

- Add ability to **mark an order as Delivered**

Business Logic (Buggy Requirement):

- If order is marked as Delivered:
 - It must NOT appear in Pending list
 - Stats must update automatically

Constraints:

- Must use reducer action
- Must not directly modify state

Edge Cases:

- Invalid order → should not update
- Already delivered → no duplicate update

Question 5 – Orders Analytics Dashboard

Route:

/stats

Requirements:

Display:

- Total valid orders
- Delivered orders
- Cancelled orders

Constraints:

- Must use .reduce()
- Must not store these values in state
- Must compute dynamically

Edge Cases:

- Ignore invalid dataset entries
- Missing status → exclude from stats

Expected Output:

```
{  
  "totalOrders": 30,  
  "deliveredOrders": 18,  
  "cancelledOrders": 6  
}
```

Testing Requirement:

```
<div data-testid="total-orders">30</div>  
<div data-testid="delivered-orders">18</div>  
<div data-testid="cancelled-orders">6</div>
```

Global State Requirement:

```
window.appState = {  
  totalOrders: 30,  
  deliveredOrders: 18,  
  cancelledOrders: 6  
};
```